NAME - Satyam Parida
Regd No - 2141016403

System Monitor Tool

Objective: Create a system monitor tool in C++ that displays real-time information about system processes, memory usage, and CPU load, similar to the 'top' command.

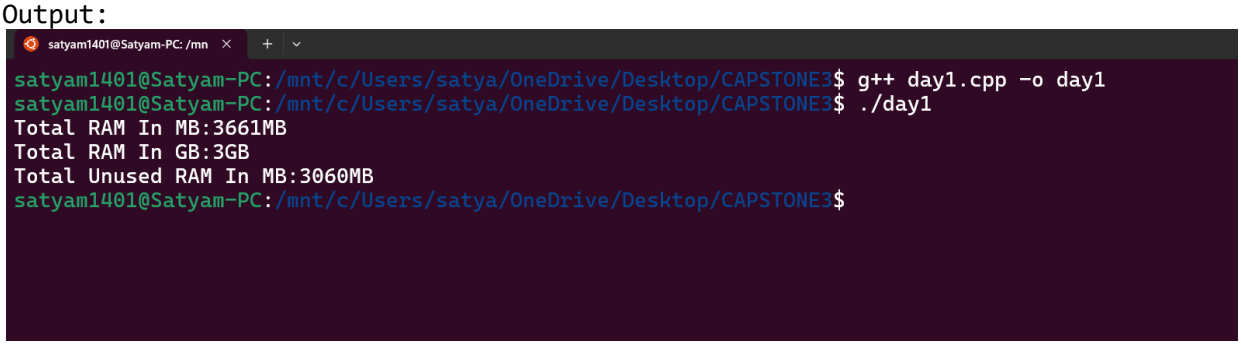
Day-wise Tasks:

Day 1: Design UI layout and gather system data using system calls.

Code:

```
#include <iostream>
using namespace std;
#include<sys/sysinfo.h>
void displayMemoryInfo()
{
struct sysinfo info;
if(sysinfo(&info)==0)
{
cout << "Total RAM In MB:" <<info.totalram/(1024*1024) << "MB\n";
cout << "Total RAM In GB:" <<info.totalram/(1024*1024*1024) <<"GB\n";
cout << "Total Unused RAM In MB:" <<info.freeram/(1024*1024) <<"MB\n";
}
}
int main(){
displayMemoryInfo();
return 0;
}
```

Output:



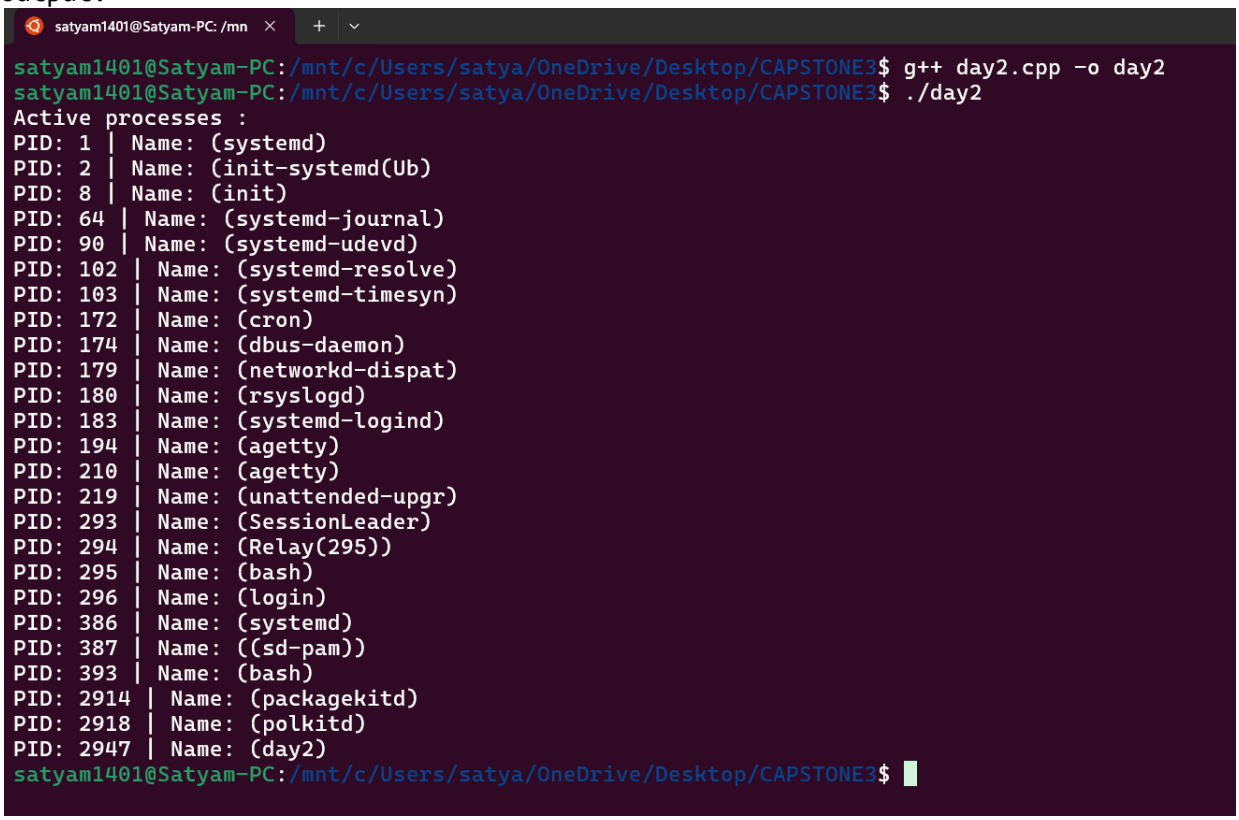
Day 2: Display process list with CPU and memory usage.

Code:

```
// cat proc/
#include <iostream>
#include <filesystem>
#include <fstream>
#include <sstream>
#include <algorithm>


namespace fs = std::filesystem;

bool isNumber(const std::string &s) {
    return !s.empty() && std::all_of(s.begin(), s.end(), ::isdigit);
}

std::string getProcessName(int pid) {
    std::ifstream file("/proc/" + std::to_string(pid) + "/stat");
    std::string line, processName;
    if (file.is_open()) {
        std::getline(file, line);
        std::istringstream ss(line);
        std::string token;
        int count = 0;
        while (ss >> token) {
            count++;
            if (count == 2) {
                processName = token;
                break;
            }
        }
    }
    return processName;
}

int main() {
    std::cout << "Active processes :" << std::endl;
    for (const auto &entry : fs::directory_iterator("/proc")) {
        if (entry.is_directory()) {
            std::string filename = entry.path().filename().string();
            if (isNumber(filename)) {
                int pid = std::stoi(filename);
                std::string processName = getProcessName(pid);
                std::cout << "PID: " << pid << " | Name: " << processName
<< std::endl;
            }
```

```
        }
    }
    return 0;
}
```

Output:



Day 3: Implement process sorting by CPU and memory usage.

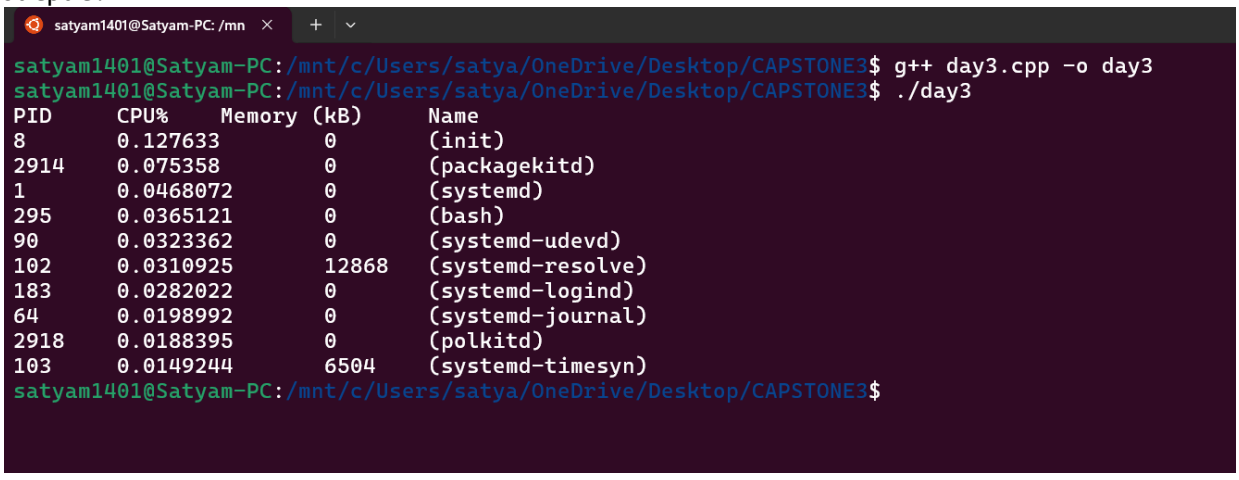
Code:

```
//Dispaly CPU & memory usage per process
//sort processes by CPU/memory usages
// Total cputime = usertime+cputime(utime+ctime)
//Total CPUTime = utime + stime;
//Calculate process cpu usage
//CPU usage = (utime+stime)/systemupdate - starttime*100;
//g++ -std=c++17 activeprocess.cpp -o activeprocess

//Memory info is in /proc/[PID]/status
//cat /proc/1/status|grep VmRSS
// VmRSS(Resident Set Size) = Actual Ram used by the process.
//student@D001-37:~/Desktop/2141011082/LOS/Day03$ cat /proc/uptime
//2456.96(total system time) 29285.05(ideal time or total system unused)
#include <iostream>
#include<fstream>//read file(/proc data)
#include<sstream>//parse data
#include<vector>//store process
#include<algorithm>//sort the process
#include <filesystem>
#include <unistd.h> // for sysconf
//sysconf(_SC_CLK_TCK)

namespace fs = std::filesystem;

struct Processinfo {
    int pid;
    std::string name;
    double cpuUsage = 0;
    long memoryUsage = 0;
};

std::string readFileValue(const std::string &path) {
    std::ifstream file(path);
    std::string value;
    if (file.is_open()) {
        std::getline(file, value);
    }
    return value;
}

double getSystemUptime() {
    std::ifstream file("/proc/uptime");
    double uptime = 0;
    if (file.is_open()) {
        file >> uptime;
    }
```

```
    return uptime;
}

Processinfo getProcessInfo(int pid, double systemUptime) {
    Processinfo proc;
    proc.pid = pid;

    std::ifstream statFile("/proc/" + std::to_string(pid) + "/stat");
    std::string line;

    long utime = 0, stime = 0, starttime = 0;

    if (statFile.is_open()) {
        std::getline(statFile, line);
        std::istringstream ss(line);
        std::string token;
        for (int i = 1; ss >> token; ++i) {
            if (i == 2) proc.name = token;
            else if (i == 14) utime = std::stol(token);
            else if (i == 15) stime = std::stol(token);
            else if (i == 22) starttime = std::stol(token);
        }
    }

    std::ifstream memFile("/proc/" + std::to_string(pid) + "/status");
    if (memFile.is_open()) {
        std::string key, value, unit;
        while (memFile >> key >> value >> unit) {
            if (key == "VmRSS:") {
                proc.memoryUsage = std::stol(value);
                break;
            }
        }
    }

    long total_time = utime + stime;
    double seconds = systemUptime - (starttime / sysconf(_SC_CLK_TCK));
    if (seconds > 0) {
        proc.cpuUsage = ((total_time / (double)sysconf(_SC_CLK_TCK)) /
seconds) * 100.0;
    }

    return proc;
}

std::vector<Processinfo> getAllprocess() {
    std::vector<Processinfo> processes;
    double systemUptime = getSystemUptime();

    for (const auto &entry : fs::directory_iterator("/proc")) {
        if (entry.is_directory()) {
            std::string filename = entry.path().filename().string();
            if (std::all_of(filename.begin(), filename.end(), ::isdigit))
{
```

```
                int pid = std::stoi(filename);
                processes.push_back(getProcessInfo(pid, systemUptime));
            }
        }
    }
    return processes;
}

void sortProcesses(std::vector<Processinfo> &processes, bool sortByCPU) {
    if (sortByCPU) {
        std::sort(processes.begin(), processes.end(), [](const
Processinfo &a, const Processinfo &b) {
            return a.cpuUsage > b.cpuUsage;
        });
    } else {
        std::sort(processes.begin(), processes.end(), [](const
Processinfo &a, const Processinfo &b) {
            return a.memoryUsage > b.memoryUsage;
        });
    }
}
int main() {
    std::vector<Processinfo> processes = getAllprocess();
    sortProcesses(processes, true); // Change to false to sort by memory
usage

    std::cout << "PID\tCPU%\tMemory (kB)\tName\n";

    for (size_t i = 0; i < std::min(processes.size(), size_t(10)); ++i) {
        std::cout << processes[i].pid << "\t"
                  << processes[i].cpuUsage << "\t"
                  << processes[i].memoryUsage << "\t"
                  << processes[i].name << "\n";
    }
    return 0;
}
```

Output:



Day 4: Add functionality to kill processes.

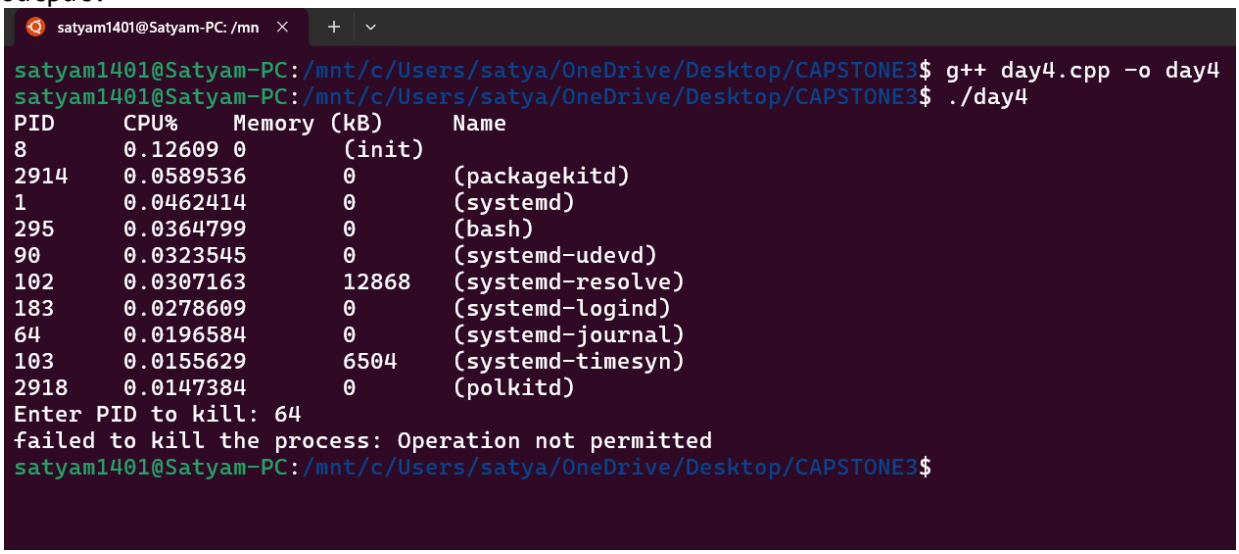
Code:

```
//Dispaly CPU & memory usage per process
//sort processes by CPU/memory usages
// Total cputime = usertime+cputime(utime+ctime)
//Total CPUTime = utime + stime;
//Calculate process cpu usage
//CPU usage = (utime+stime)/systemupdate - starttime*100;
//g++ -std=c++17 activeprocess.cpp -o activeprocess

//Memory info is in /proc/[PID]/status
//cat /proc/1/status|grep VmRSS
// VmRSS(Resident Set Size) = Actual Ram used by the process.
//student@D001-37:~/Desktop/2141011082/LOS/Day03$ cat /proc/uptime
//2456.96(total system time) 29285.05(ideal time or total system unused)
#include <iostream>
#include<fstream>//read file(/proc data)
#include<sstream>//parse data
#include<vector>//store process
#include<algorithm>//sort the process
#include <filesystem>
#include <unistd.h> // for sysconf
#include <csignal>
//sysconf(_SC_CLK_TCK)

namespace fs = std::filesystem;

struct Processinfo {
    int pid;
    std::string name;
    double cpuUsage = 0;
    long memoryUsage = 0;
};

std::string readFileValue(const std::string &path) {
    std::ifstream file(path);
    std::string value;
    if (file.is_open()) {
        std::getline(file, value);
    }
    return value;
}

double getSystemUptime() {
    std::ifstream file("/proc/uptime");
    double uptime = 0;
    if (file.is_open()) {
        file >> uptime;
    }
```

```
    return uptime;
}

Processinfo getProcessInfo(int pid, double systemUptime) {
    Processinfo proc;
    proc.pid = pid;

    std::ifstream statFile("/proc/" + std::to_string(pid) + "/stat");
    std::string line;

    long utime = 0, stime = 0, starttime = 0;

    if (statFile.is_open()) {
        std::getline(statFile, line);
        std::istringstream ss(line);
        std::string token;
        for (int i = 1; ss >> token; ++i) {
            if (i == 2) proc.name = token;
            else if (i == 14) utime = std::stol(token);
            else if (i == 15) stime = std::stol(token);
            else if (i == 22) starttime = std::stol(token);
        }
    }

    std::ifstream memFile("/proc/" + std::to_string(pid) + "/status");
    if (memFile.is_open()) {
        std::string key, value, unit;
        while (memFile >> key >> value >> unit) {
            if (key == "VmRSS:") {
                proc.memoryUsage = std::stol(value);
                break;
            }
        }
    }

    long total_time = utime + stime;
    double seconds = systemUptime - (starttime / sysconf(_SC_CLK_TCK));
    if (seconds > 0) {
        proc.cpuUsage = ((total_time / (double)sysconf(_SC_CLK_TCK)) /
seconds) * 100.0;
    }

    return proc;
}

std::vector<Processinfo> getAllprocess() {
    std::vector<Processinfo> processes;
    double systemUptime = getSystemUptime();

    for (const auto &entry : fs::directory_iterator("/proc")) {
        if (entry.is_directory()) {
            std::string filename = entry.path().filename().string();
            if (std::all_of(filename.begin(), filename.end(), ::isdigit))
{
```

```
                int pid = std::stoi(filename);
                processes.push_back(getProcessInfo(pid, systemUptime));
            }
        }
    }
    return processes;
}

void sortProcesses(std::vector<Processinfo> &processes, bool sortByCPU) {
    if (sortByCPU) {
        std::sort(processes.begin(), processes.end(), [](const
Processinfo &a, const Processinfo &b) {
            return a.cpuUsage > b.cpuUsage;
        });
    } else {
        std::sort(processes.begin(), processes.end(), [](const
Processinfo &a, const Processinfo &b) {
            return a.memoryUsage > b.memoryUsage;
        });
    }
}

int main() {
    std::vector<Processinfo> processes = getAllprocess();
    sortProcesses(processes, true); // Change to false to sort by memory
usage

    std::cout << "PID\tCPU%\tMemory (kB)\tName\n";

    for (size_t i = 0; i < std::min(processes.size(), size_t(10)); ++i) {
        std::cout << processes[i].pid << "\t"
                  << processes[i].cpuUsage << "\t"
                  << processes[i].memoryUsage << "\t"
                  << processes[i].name << "\n";
    }
    int targetPid;
    std::cout <<"Enter PID to kill: ";
    std::cin >> targetPid;

    //Signature : int kill(pid_t pid, int sig);
    //pid: Process ID
    //sig: Signal to send(SIGKILL, SIGTERM, etc.)
    if(targetPid > 0){
           if(kill(targetPid, SIGKILL)==0){
           std::cout << "Process " << targetPid << "terminated
Successfully";
           }
           else{
           perror("failed to kill the process");
           }

    return 0;
}
}
```

Output:


satyam1401@Satyam-PC: /mn
satyam1401@Satyam-PC:/mnt/c/Users/satya/OneDrive/Desktop/CAPSTONE3$ g++ day4.cpp -o day4
satyam1401@Satyam-PC:/mnt/c/Users/satya/OneDrive/Desktop/CAPSTONE3$ ./day4
PID CPU% Memory (kB) Name
8 0.12609 0 (init)
2914 0.0589536 0 (packagekitd)
1 0.0462414 0 (systemd)
295 0.0364799 0 (bash)
90 0.0323545 0 (systemd-udevd)
102 0.0307163 12868 (systemd-resolve)
183 0.0278609 0 (systemd-logind)
64 0.0196584 0 (systemd-journal)
103 0.0155629 6504 (systemd-timesyn)
2918 0.0147384 0 (polkitd)
Enter PID to kill: 64
failed to kill the process: Operation not permitted
satyam1401@Satyam-PC:/mnt/c/Users/satya/OneDrive/Desktop/CAPSTONE3$

Day 5: Implement real-time update feature to refresh data every few seconds.

Code:

```
//Dispaly CPU & memory usage per process
//sort processes by CPU/memory usages
// Total cputime = usertime+cputime(utime+ctime)
//Total CPUTime = utime + stime;
//Calculate process cpu usage
//CPU usage = (utime+stime)/systemupdate - starttime*100;
//g++ -std=c++17 activeprocess.cpp -o activeprocess

//Memory info is in /proc/[PID]/status
//cat /proc/1/status|grep VmRSS
// VmRSS(Resident Set Size) = Actual Ram used by the process.
//student@D001-37:~/Desktop/2141011082/LOS/Day03$ cat /proc/uptime
//2456.96(total system time) 29285.05(ideal time or total system unused)
#include <iostream>
#include<fstream>//read file(/proc data)
#include<sstream>//parse data
#include<vector>//store process
#include<algorithm>//sort the process
#include <filesystem>
#include <unistd.h> // for sysconf
//sysconf(_SC_CLK_TCK)
#include<csignal>//for kill() and SIGKILL
#include<thread>
#include<chrono>

namespace fs = std::filesystem;
using namespace std;

struct Processinfo {
    int pid;
    std::string name;
    double cpuUsage = 0;
    long memoryUsage = 0;
};

std::string readFileValue(const std::string &path) {
    std::ifstream file(path);
    std::string value;
    if (file.is_open()) {
        std::getline(file, value);
    }
    return value;
}

double getSystemUptime() {
    std::ifstream file("/proc/uptime");
    double uptime = 0;
```

```
    if (file.is_open()) {
        file >> uptime;
    }
    return uptime;
}

Processinfo getProcessInfo(int pid, double systemUptime) {
    Processinfo proc;
    proc.pid = pid;

    std::ifstream statFile("/proc/" + std::to_string(pid) + "/stat");
    std::string line;

    long utime = 0, stime = 0, starttime = 0;

    if (statFile.is_open()) {
        std::getline(statFile, line);
        std::istringstream ss(line);
        std::string token;
        for (int i = 1; ss >> token; ++i) {
            if (i == 2) proc.name = token;
            else if (i == 14) utime = std::stol(token);
            else if (i == 15) stime = std::stol(token);
            else if (i == 22) starttime = std::stol(token);
        }
    }

    std::ifstream memFile("/proc/" + std::to_string(pid) + "/status");
    if (memFile.is_open()) {
        std::string key, value, unit;
        while (memFile >> key >> value >> unit) {
            if (key == "VmRSS:") {
                proc.memoryUsage = std::stol(value);
                break;
            }
        }
    }

    long total_time = utime + stime;
    double seconds = systemUptime - (starttime / sysconf(_SC_CLK_TCK));
    if (seconds > 0) {
        proc.cpuUsage = ((total_time / (double)sysconf(_SC_CLK_TCK)) /
seconds) * 100.0;
    }

    return proc;
}

std::vector<Processinfo> getAllprocess() {
    std::vector<Processinfo> processes;
    double systemUptime = getSystemUptime();

    for (const auto &entry : fs::directory_iterator("/proc")) {
        if (entry.is_directory()) {
```

```
            std::string filename = entry.path().filename().string();
            if (std::all_of(filename.begin(), filename.end(), ::isdigit))
{
                int pid = std::stoi(filename);
                processes.push_back(getProcessInfo(pid, systemUptime));
            }
        }
    }
    return processes;
}

void sortProcesses(std::vector<Processinfo> &processes, bool sortByCPU) {
    if (sortByCPU) {
        std::sort(processes.begin(), processes.end(), [](const
Processinfo &a, const Processinfo &b) {
            return a.cpuUsage > b.cpuUsage;
        });
    } else {
        std::sort(processes.begin(), processes.end(), [](const
Processinfo &a, const Processinfo &b) {
            return a.memoryUsage > b.memoryUsage;
        });
    }
}

int main() {
    char input;
    while(true){
        system("clear");
        vector<Processinfo> processes = getAllprocess();
        sortProcesses(processes, true); // Change to false to sort by
memory usage

        cout << "PID\tCPU%\tMemory (kB)\tName\n";
        for (size_t i = 0; i < min(processes.size(), size_t(10)); ++i) {
            cout << processes[i].pid << "\t"
            << processes[i].cpuUsage << "\t"
            << processes[i].memoryUsage << "\t"
            << processes[i].name << "\n";
        }
        int targetPid;
        cout <<"Enter PID to Kill : " << endl;
        cin >> targetPid;
        //Signature : int kill(pid_t,int sig)
        if(targetPid){
            if(kill(targetPid,SIGKILL)==0){
                cout << "Process " << targetPid << " terminated
successfully" << endl;
            }
            else{
                perror("Failed to kill Process");
            }
        }
        cout <<"\nPress 'q' to quit or Enter to refresh : ";
```

```
        cin.ignore();
        input = getchar();
        if(input == 'q' || input == 'Q')
            break;
        std::this_thread::sleep_for(std::chrono::seconds(2));
    }
    return 0;
}
```

Output: